

SELinux, Firewalls & Ports

A practical field guide: acronyms, default ports, port limits,
client-server communication, firewall commands, Cockpit, and real-world scenarios

Prepared for homelab / RHEL & ESXi practice

1. Key Acronyms

Acronym	Full Form	Relevance
SELinux	Security-Enhanced Linux	Kernel security module enforcing Mandatory Access Control (MAC)
MAC	Mandatory Access Control	SELinux's model — policy decides access, not just file owner
DAC	Discretionary Access Control	Traditional Linux permission model (rwx, owner/group)
TCP	Transmission Control Protocol	Connection-oriented, reliable transport (used by SSH, HTTP, etc.)
UDP	User Datagram Protocol	Connectionless transport (used by DNS queries, DHCP, etc.)
IANA	Internet Assigned Numbers Authority	Body that assigns and registers well-known port numbers
NIC	Network Interface Card	Physical/virtual network adapter
NAT	Network Address Translation	Maps private IPs to public IPs, often reusing ports
SYN/ACK	Synchronize / Acknowledge	Flags used in the TCP three-way handshake
SSH	Secure Shell	Encrypted remote login protocol, default port 22
FQDN	Fully Qualified Domain Name	Complete domain name, e.g. satellite.homelab.local
firewalld	Dynamic Firewall Daemon	Default firewall management service on RHEL/Fedora
iptables/nftables	Packet filtering frameworks	Underlying packet-filter engines firewalld talks to
Cockpit	Web-based server console	Browser-based admin UI for Linux servers, port 9090
SELinux Boolean	Toggleable policy switch	on/off switches that change SELinux behaviour without editing policy
Port Context	SELinux port label	SELinux type bound to a TCP/UDP port, e.g. http_port_t

2. SELinux + Firewall — How They Relate

SELinux and the firewall solve two different problems, and people often mix them up. The **firewall** (firewalld/iptables/nftables) decides whether a network packet is allowed to reach a port at all — it works at the network/transport layer. **SELinux** decides, once a process is running, what that process is allowed to do — including which port types it is allowed to bind to. So a service can fail for two independent reasons: the firewall is blocking the port, or SELinux is blocking the process from using that port even though the firewall allowed it.

Note: A very common real-world symptom: you open a port in firewalld, the service still doesn't respond externally, and the actual cause is SELinux denying the daemon from binding a non-default port (check with `ausearch -m avc -ts recent`).

3. Default Ports for Common Services

These are the IANA-registered 'well-known' ports most services use out of the box:

Service	Port(s)	Protocol	Purpose
FTP (data)	20	TCP	File Transfer Protocol — data channel
FTP (control)	21	TCP	File Transfer Protocol — command channel
SSH	22	TCP	Secure remote shell / SCP / SFTP
Telnet	23	TCP	Unencrypted remote login (legacy)
SMTP	25	TCP	Sending email between mail servers

Service	Port(s)	Protocol	Purpose
DNS	53	TCP/UDP	Domain name resolution (BIND, etc.)
DHCP	67/68	UDP	Dynamic IP address assignment
TFTP	69	UDP	Trivial file transfer (PXE boot, etc.)
HTTP	80	TCP	Unencrypted web traffic
Kerberos	88	TCP/UDP	Authentication (used by FreeIPA/AD)
POP3	110	TCP	Retrieving email
NTP	123	UDP	Network time synchronization
IMAP	143	TCP	Retrieving/managing email on server
SNMP	161/162	UDP	Network device monitoring
LDAP	389	TCP/UDP	Directory services (FreeIPA, AD)
HTTPS	443	TCP	Encrypted web traffic (TLS)
SMB/CIFS	445	TCP	Windows file sharing
SMTPS	465	TCP	Encrypted SMTP submission
Syslog	514	UDP	Centralized system logging
LDAPS	636	TCP	Encrypted LDAP
IMAPS	993	TCP	Encrypted IMAP
POP3S	995	TCP	Encrypted POP3
MySQL/MariaDB	3306	TCP	Database connections
RDP	3389	TCP	Windows Remote Desktop
PostgreSQL	5432	TCP	Database connections
VNC	5900+	TCP	Remote desktop (console access)
Foreman/Satellite	443, 8443	TCP	Web UI + Smart-proxy
Cockpit	9090	TCP	Web-based server management console
Ansible (control node)	22 (uses SSH)	TCP	Ansible has no daemon port — it rides over SSH

4. How Many Ports Exist — and Why 65,535?

Every TCP and UDP header reserves a fixed 16-bit field for the port number. A 16-bit binary field can represent $2^{16} = 65,536$ distinct values, numbered 0 through 65,535. That gives **65,536 ports total per protocol, per IP address** — usually quoted as "65,535 ports" because port 0 is reserved and not actually used for connections.

Range	Numbers	Also called	Typical use
Well-known ports	0 – 1023	System / privileged ports	Standard services (SSH, HTTP, DNS...). Require root/CAP_NET_BIND
Registered ports	1024 – 49151	User ports	Registered with IANA by app vendors (e.g. MySQL 3306, Cockpit 9090)
Dynamic / private ports	49152 – 65535	Ephemeral ports	Assigned temporarily by the OS to the client side of a connection.

Can we increase the number of ports beyond 65,535?

No — not for a single IP address. This is not a configuration limit that an admin can raise; it is a hard limit built into the TCP/UDP protocol specification itself (the port field is fixed at 16 bits in every packet header ever sent on the internet). Changing it would require rewriting TCP/UDP everywhere, which is why it hasn't happened in 40+ years.

What you actually do when 65,535 ports per IP isn't enough:

- **Add more IP addresses** to the same server (each IP gets its own full 65,536-port range).
- **Use application-layer multiplexing** — e.g. one HTTPS port (443) serving hundreds of virtual hosts via SNI/Host headers, instead of a unique port per app.
- **Use a reverse proxy / load balancer** (Nginx, HAProxy) that maps many internal services onto one public port.
- **Use IPv6** if the real constraint is address exhaustion rather than ports — but the 65,536-port-per-address limit still applies under IPv6 too.

Note: So the number you were thinking of — 65,535 — is correct, and it cannot be raised. What can scale instead is the number of IP addresses or the number of services multiplexed behind one port.

5. Ports on a Bare-Metal Server vs. a Laptop

This is a common point of confusion: ports are not a piece of hardware you are issued, like RAM or a MAC address. The 65,536-port range is defined by the TCP/IP protocol itself and is identical on every device that runs a TCP/IP stack — a brand-new bare-metal server, a laptop, a Raspberry Pi, or a phone all have the exact same 65,536 potential ports available per IP address.

Device	Total possible ports	Ports actually 'open'	What determines it
Bare-metal server (fresh install)	65,536 (0-65535)	Very few — typically only SSH (22)	Only ports with a listening service AND allowed by firewall
Laptop (fresh install)	65,536 (0-65535)	Often none from the outside	Laptops usually run no server daemons and firewalls
Server after installing services	65,536 (0-65535)	One per running daemon (e.g. 22, 443, 9090)	Each service has its own port; firewall must allow

So: the *capacity* (65,536) never changes with hardware. What changes is how many of those ports have a service *listening* on them, and how many the firewall allows through. You can check what's actually listening with:

```
ss -tulnp # modern replacement for netstat
sudo firewall-cmd --list-ports
sudo firewall-cmd --list-services
```

6. What Happens Port-Wise When Your Laptop Talks to a Server

When your laptop sends a request to a server (say, SSH to a RHEL box, or opening Cockpit's web UI), two different ports are involved on two different machines:

- **On the server**, the destination port is fixed and well-known — e.g. 22 for SSH, 443 for HTTPS, 9090 for Cockpit. The service daemon (sshd, httpd, cockpit.socket) is already listening on that port.
- **On your laptop**, the OS does NOT reuse that same port. It picks a random free port from the **ephemeral range (typically 49152-65535, though Linux often uses 32768-60999)** as the source port for that one connection.

Step-by-step (TCP three-way handshake)

- **1. SYN** — Laptop (source port e.g. 51342) sends a SYN packet to Server:22 asking to open a connection.
- **2. SYN-ACK** — Server replies from port 22 back to the laptop's port 51342, acknowledging and proposing its own sequence number.

- **3. ACK** — Laptop acknowledges back to Server:22. The connection is now established, uniquely identified by the 4-tuple (source IP, source port, dest IP, dest port).
- **4. Data exchange** — All further packets (your typed commands, the returned terminal output, or the web page HTML) flow between exactly that (laptop:51342 <-> server:22) pair until the connection closes.
- **5. Response routing** — The server's reply always goes back to the same ephemeral port it received the request from — that's how your laptop knows which browser tab / SSH session the reply belongs to, even if you have many connections open at once.

Note: This is why a firewall rule on the server only needs to open the well-known port (22, 443, 9090...) inbound. It does not need to open the client's ephemeral port — return traffic for an already-established connection is automatically permitted by connection tracking (the 'RELATED,ESTABLISHED' rule).

7. Firewall Commands (RHEL/CentOS/Fedora — firewalld)

7.1 Service control

```
sudo systemctl start firewalld # start the firewall now
sudo systemctl stop firewalld # stop the firewall now
sudo systemctl enable firewalld # start automatically on boot
sudo systemctl disable firewalld # do not start on boot
sudo systemctl status firewalld # check if it's running
sudo systemctl restart firewalld # restart the service
```

7.2 Checking state

```
sudo firewall-cmd --state # running / not running
sudo firewall-cmd --get-zones # list all available zones
sudo firewall-cmd --get-default-zone # show the active default zone
sudo firewall-cmd --list-all # full detail of the default zone
sudo firewall-cmd --list-all-zones # detail of every zone
```

7.3 Opening / closing a port or service

```
# Open by port number (runtime only, lost on reboot)
sudo firewall-cmd --add-port=8080/tcp

# Open by port number PERMANENTLY, then apply it
sudo firewall-cmd --permanent --add-port=8080/tcp
sudo firewall-cmd --reload

# Open by known service name instead of raw port
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --permanent --add-service=https
sudo firewall-cmd --permanent --add-service=cockpit
sudo firewall-cmd --reload

# Remove / close a port or service
sudo firewall-cmd --permanent --remove-port=8080/tcp
sudo firewall-cmd --permanent --remove-service=http
sudo firewall-cmd --reload
```

7.4 Zones and rich rules

```
# Assign an interface to a zone
sudo firewall-cmd --zone=internal --change-interface=eth1 --permanent

# Allow a port only from one specific subnet (rich rule)
sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
source address="192.168.1.0/24" port port="3306" protocol="tcp" accept'
sudo firewall-cmd --reload
```

7.5 Legacy iptables equivalents (for reference)

```
sudo iptables -L -n -v # list current rules
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT # allow SSH
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT # allow HTTP
sudo iptables -D INPUT -p tcp --dport 80 -j ACCEPT # delete that rule
sudo service iptables save # persist rules (older RHEL)
```

7.6 SELinux port labelling (semanage)

If a service needs to bind a non-default port, opening it in the firewall is not enough — SELinux must also be told that this port is allowed for that service's type.

```
sudo semanage port -l | grep http # list ports SELinux allows for httpd
sudo semanage port -a -t http_port_t -p tcp 8081 # add port 8081 as http_port_t
sudo semanage port -d -t http_port_t -p tcp 8081 # remove that mapping

getsebool -a | grep httpd # list httpd-related booleans
sudo setsebool -P httpd_can_network_connect on # persist a boolean change
sudo restorecon -Rv /var/www/html # fix mislabeled file contexts
```

8. Cockpit

Cockpit is Red Hat's built-in, browser-based server management console. It lets you monitor CPU/memory/disk, manage services, view logs, manage users, configure networking, and even open a terminal — all from a web browser, without needing a separate GUI installed on the server.

Item	Value
Default port	9090 (TCP, HTTPS with a self-signed cert by default)
Service/socket name	cockpit.socket
Package name (RHEL/CentOS)	cockpit
Access URL	https://server-ip-or-hostname:9090

Install and enable Cockpit

```
sudo dnf install cockpit -y
sudo systemctl enable --now cockpit.socket
sudo firewall-cmd --permanent --add-service=cockpit
sudo firewall-cmd --reload

# Verify it's listening
ss -tulnp | grep 9090

# Disable it again if not needed
sudo systemctl disable --now cockpit.socket
sudo firewall-cmd --permanent --remove-service=cockpit
sudo firewall-cmd --reload
```

Note: Cockpit uses your normal Linux login credentials — there's no separate Cockpit password to manage.

9. Real-World Scenarios

Scenario 1 — Hosting a web server (Apache/Nginx)

You install httpd, start it, but external browsers can't reach the page.

```
sudo dnf install httpd -y
sudo systemctl enable --now httpd
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --permanent --add-service=https
sudo firewall-cmd --reload
sudo setsebool -P httpd_can_network_connect on # if the app calls out to a DB/API
```

Scenario 2 — Moving SSH off the default port 22 for security

You change sshd to listen on 2222 to reduce automated attacks — this needs BOTH firewalld and SELinux updated, or the daemon won't even start.

```
sudo semanage port -a -t ssh_port_t -p tcp 2222
sudo firewall-cmd --permanent --add-port=2222/tcp
sudo firewall-cmd --reload
sudo sed -i 's/#Port 22/Port 2222/' /etc/ssh/sshd_config
sudo systemctl restart sshd
```

Scenario 3 — Restricting database access to only the app server

MySQL should only be reachable from your application subnet, not the whole internet — use a rich rule instead of a blanket --add-service.

```
sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
source address="10.0.1.0/24" port port="3306" protocol="tcp" accept'
sudo firewall-cmd --reload
```

Scenario 4 — Deploying an app on a custom port (e.g. Tomcat on 8080)

A Java app fails to bind port 8081 with 'Permission denied' even though the firewall is open — classic SELinux port-labelling issue.

```
sudo semanage port -a -t http_port_t -p tcp 8081
sudo firewall-cmd --permanent --add-port=8081/tcp
sudo firewall-cmd --reload
sudo systemctl restart tomcat
```

Scenario 5 — Setting up Cockpit for remote homelab management (your ESXi/RHEL homelab)

You want to manage a RHEL VM (e.g. your satellite.homelab.local Foreman box) visually from your laptop's browser instead of SSH-only.

```
sudo dnf install cockpit cockpit-storaged cockpit-networkmanager -y
sudo systemctl enable --now cockpit.socket
sudo firewall-cmd --permanent --add-service=cockpit
sudo firewall-cmd --reload
# From your laptop browser:
# https://satellite.homelab.local:9090
```

Quick recap: 65,536 ports exist per IP address on any device — this is a fixed 16-bit protocol limit and cannot be increased. Firewalld controls what's allowed in; SELinux controls what a process is allowed to bind to; both must agree for a service to be reachable.